Relatório de Tecnologias
Nó Soberano - PIDE / FAPESPA

o papel de cada tecnologia na arquitetura
e a resposta em uma frase para cada 'o que isso faz?'

Stack real da malha de dois nós (deploy/malha/) ?
a mesma imagem em cada instituição; muda o papel,
não a stack.

1. Visão geral da stack

Cada nó soberano é a MESMA composição de serviços. O que muda entre instituições é o papel na tramitação (origem ou destino), nunca a stack. Esta é a arquitetura-alvo: cada órgão opera esta composição na própria infraestrutura.

Camada	Tecnologia	Função no nó
API	FastAPI (Python)	tramitações, upload, recebimentos, acks, verificação
Banco	PostgreSQL	metadados, trilha hash-chain, outbox, replay_nonces ? SEM bytes
Fila	RabbitMQ (TLS)	esteira assíncrona de anonimização ? quorum + DLQ
Storage	MinIO (HTTPS)	objetos ? sempre ciphertext E2E em repouso
Borda	nginx + mTLS	identidade institucional por certificado de cliente
Segredos	Vault	custódia da chave E2E ? AppRole, rotação, revogação
Worker	Python + MotorIA	anonimização via porta permutável
Relay	Python	outbox -> broker
Cripto	ChaCha20-Poly1305, Ed25519, SHA-256	E2E nos 3 pontos de I/O; assinaturas; integridade
Plataforma	Docker Compose, Alembic, pytest	malha de 2 nós; migrações; suíte smoke/e2e

Regra que atravessa tudo: o binário do documento NUNCA entra em banco, envelope, broker ou resposta de API ? só metadados + SHA-256 circulam entre os nós.

2. Os serviços do nó ? um a um

FastAPI (a API)

Papel na arquitetura: a superfície pública do nó ? todas as operações de tramitação passam por ela.

O que é: framework web Python assíncrono. Recebe criação de tramitação, upload multipart, recebimento de envelope, acks e o endpoint de verificação da cadeia.

Por que ela: mesma linguagem do worker e do motor ? uma base só de código; async aguenta I/O concorrente sem complexidade.

Onde ver: app/main.py, app/service.py, app/deps.py.

> *>> é a porta do nó: tudo que a instituição ou outra instituição pede ao sistema entra pela API ? e ela aplica as regras antes de tocar storage ou banco.*

PostgreSQL (o banco)

Papel na arquitetura: guarda só metadados e evidências ? nunca o documento.

O que é: registros de tramitação, trilha de auditoria (hash-chain), outbox de eventos, replay_nonces e o cofre de pseudônimos.

Por que ela: transações: o upload grava objeto + trilha + outbox numa ÚNICA transação ? não existe 'arquivou mas não registrou'. E o gatilho append-only barra UPDATE/DELETE na trilha no nível do banco.

Onde ver: alembic/versions/ (migrações ? 0007 = append-only); app/repository.py.

> *>> o banco é o livro-razão: metadados, hashes e a corrente de auditoria ? e por desenho não existe sequer coluna para guardar bytes de documento.*

RabbitMQ (a fila)

Papel na arquitetura: a esteira assíncrona ? desacopla a API do processamento pesado de anonimização.

O que é: broker de mensagens só em TLS (porta 5671). Fila quorum com DLQ: evento de 'documento arquivado' dispara o worker sem a chamada de upload ficar esperando.

Por que ela: anonimização pode demorar ? a API responde na hora e o processamento continua em segundo plano. Fila quorum sobrevive a restart; DLQ guarda o que falhar de vez.

Onde ver: deploy/malha/rabbitmq.conf (TLS-only); app/worker.py (consumidor); app/relay.py (produtor via outbox).

> *>> é a correia interna do nó: o evento sai da API, viaja cifrado pela fila e acorda o worker ? tudo dentro do domínio da instituição.*

MinIO (o storage)

Papel na arquitetura: o cofre de objetos da instituição ? guarda o documento e depois o anonimizado.

O que é: object storage compatível com S3, servido em HTTPS. Dois buckets por nó: <org>-docs e <org>-docs-anonimizados.

Por que ela: soberania física: cada instituição é dona do próprio storage. E em repouso só existe ciphertext ? o objeto cru começa com a magia E2 (0xE2 0x02), legível só dentro do processo.

Onde ver: app/storage.py (ArmazenamentoCifrado); prova no terminal: 'objeto cru no MinIO' na demo.

> *>> é o cofre da instituição: quem olha o MinIO por dentro vê só ciphertext ? o documento em claro nunca repousa em lugar nenhum.*

nginx + mTLS (a borda)

Papel na arquitetura: a fronteira institucional ? quem bate na porta precisa de certificado de cliente válido.

O que é: proxy reverso que TERMINA o mTLS: verifica o certificado da instituição cliente e injeta X-Cliente-Orgao. Aplica rate limit e HSTS.

Por que ela: identidade institucional por criptografia de certificado ? substituiu JWT (reprovado: token em browser é vulnerável a XSS). mTLS identifica o ÓRGÃO, não um usuário.

Onde ver: deploy/malha: nginx.conf + certs gerados por scripts/gerar_certs_mtls.py.

> *>> é a portaria do prédio: só entra quem apresenta credencial de instituição reconhecida ? e a borda já carimba de que órgão é a requisição.*

Vault (os segredos)

Papel na arquitetura: a custódia das chaves ? a chave E2E mora lá, não no código nem em arquivo.

O que é: servidor de segredos (KV v2): a API e o worker leem a chave E2E via AppRole ? login com role_id + secret_id, token curto de 60s renovável. Suporta rotação (atual + anterior_N) e revogação.

Por que ela: separar a chave do código: dá para girar a chave sem trocar deploy, revogar sem recompilar, e auditar quem leu o que. E2E_FONTE=env|vault ? env só para dev.

Onde ver: app/cofre.py; deploy/malha/vault-init.sh (policy e2e-leitura + role no-a).

> *>> é o cofre de chaves do prédio: a chave que decifra os envelopes não fica escrita em lugar nenhum do código ? cada serviço pede emprestada ao Vault e ela tem prazo de validade.*

Worker + porta MotorIA

Papel na arquitetura: o anonimizador ? roda DENTRO do nó, o documento nunca sai para ser tratado.

O que é: processo que consome a fila, baixa a cópia cifrada, aplica o motor e publica no bucket anonimizados ? depois do gate de PII residual (fail-closed: sobrou PII, não grava).

Por que ela: MotorIA é uma PORTA permutável: anonimizar(bytes)->bytes. Trocar o motor (regex, modelo medido, NER discriminativo futuro) não toca worker, fila ou buckets.

Onde ver: app/worker.py; implementações: DemoMotorIA (demo), app/motor_ollama.py (POC medida).

> *>> é o funcionário que trata o documento no arquivo do próprio órgão: o motor é peça trocável ? a esteira ao redor não muda.*

Relay (o outbox)

Papel na arquitetura: a ponte transacional entre banco e fila.

O que é: varre a tabela outbox e publica no RabbitMQ ? o evento é gravado junto com a operação na mesma transação e o relay garante a entrega ao broker (com retry).

Por que ela: resolve o problema clássico: sem outbox, 'commitou no banco mas caiu antes de avisar a fila' perderia o evento. Com outbox, gravar e anunciar são atômicos.

Onde ver: app/relay.py; migração 0005 (outbox retry).

> *>> é o carteiro interno: o que o banco registra, o relay garante que chega à fila ? nenhum evento se perde entre a gravação e o processamento.*

3. As primitivas de segurança

Não são 'serviços' mas são o que a plateia pergunta quando vê 'E2E', 'assinatura' e 'hash' na tela.

ChaCha20-Poly1305 (o E2E)

Papel na arquitetura: a blindagem ponta a ponta ? tudo que atravessa ou repousa vai dentro de quadros AEAD.

O que é: cifra autenticada: cada quadro é 0xE2|0x02|nonce|len|ts_ms|ct ? o timestamp entra no AAD (adulterá-lo quebra o selo). Cobre os 3 pontos de I/O: request/response HTTP, mensagens do broker e objetos no storage.

Por que ela: AEAD = confidencialidade + integridade no mesmo selo; ChaCha20 é rápido em software puro (sem AES-NI). O formato versionado permite evoluir o quadro.

Onde ver: app/crypto_e2e.py (selar/abrir), app/middleware_e2e.py (HTTP), PublicadorRabbitMQ (broker), ArmazenamentoCifrado (storage).

> *>> é o envelope lacrado de ponta a ponta: mesmo que alguém capture o pacote no fio ou abra o MinIO, vê só ciphertext ? e qualquer alteração estoura o lacre.*

Ed25519 (as assinaturas)

Papel na arquitetura: a prova de autoria ? envelopes e acks são assinados pela chave da instituição.

O que é: assinatura digital de curva elíptica: A assina TODOS os campos do envelope; B verifica contra a chave pública de A antes de persistir. O ack é assinado por B e verificado por A.

Por que ela: determinística, rápida, chaves curtas ? e fail-closed por construção: uma vírgula adulterada invalida tudo.

Onde ver: app/assinatura.py; chaves ORG_SIGN_*/ORGAO_PUBKEYS_* no .env da malha (gitignored).

> *>> é o selo de cera da instituição: quem recebe sabe DE QUEM veio e que ninguém mexeu ? por isso o envelope adulterado da demo é jogado fora.*

SHA-256 (integridade + cadeia)

Papel na arquitetura: a impressão digital do documento e o elo da corrente de auditoria.

O que é: hash criptográfico: declarado no criar, conferido no upload, revalidado no download e recomputado por B no próprio storage antes de assinar. Na trilha, cada elo é sha256(elo anterior | evento).

Por que ela: determinístico e público ? qualquer ponta recomputa e confere sem confiar na outra. Encadear transforma uma lista de eventos em uma corrente: mexer num elo quebra todos os seguintes.

Onde ver: app/repository.py (encadear_uma_vez); GET /verificar rejoga do GENESIS.

> *>> é a impressão digital: não prova que você tem o documento certo porque alguém disse ? prova porque os 64 hexadecimais batem, e cada um pode conferir sozinho.*

replay_nonces (anti-replay)

Papel na arquitetura: o carimbo único ? um pacote cifrado só vale uma vez.

O que é: tabela no Postgres onde cada nonce de quadro E2E é reivindicado atomicamente (PK) antes de processar; janela de frescura E2E_JANELA_MS = 300s.

Por que ela: capturar e reentregar o MESMO pacote não pode repetir a operação ? o claim atômico garante isso até com réplicas. AMQP só usa frescura (redelivery legítima repete o envelope).

Onde ver: middleware_e2e (claim); migração de replay_nonces; passo 'replay' da demo prova 200 -> 400.

> *>> é o carimbo de correio: o pacote chega, recebe o carimbo único ? a fotocópia dele depois é rejeitada porque esse carimbo já foi gasto.*

4. Plataforma e motor ? o que sustenta

Docker Compose (a malha)

Papel na arquitetura: a bancada que prova tudo: 2 nós reais, ~16 contêineres.

O que é: cada nó = api + postgres + rabbitmq + minio + worker + relay + nginx; mais vault e o contêiner peer que dirige a demo. Redes separadas: borda (mTLS) e interna.

Por que ela: reproduzível: docker compose up e a arquitetura inteira existe ? com TLS real em todos os links internos (postgres ssl, rabbitmq 5671, minio https, vault https).

Onde ver: deploy/malha/compose.malha.yaml (+ compose.demo.yaml para o motor da demo).

> *>> é a maquete que funciona de verdade: não é desenho da arquitetura, é a arquitetura executando ? cada órgão real rodaria essa mesma composição na sua infra.*

Alembic (migrações)

Papel na arquitetura: a evolução versionada do banco.

O que é: migrações incrementais: 0005 = outbox retry, 0007 = gatilho append-only + replay_nonces.

Por que ela: o schema não é arte manual: sobe junto com a aplicação (alembic upgrade head no boot da api).

Onde ver: alembic/versions/.

> *>> é o diário de obras do banco: cada mudança de estrutura é uma migração versionada e aplicada na subida.*

Motor de anonimização ? a ressalva honesta

Papel na arquitetura: a peça que remove PII ? trocável por desenho.

O que é: na DEMO roda DemoMotorIA (regex LGPD determinística) ? a anonimização fica VISÍVEL e repetível ao vivo. O candidato medido é a cascata regex+Ollama qwen2.5: NER exact F1 0,953, reidentificação 0/360 ? no corpus SINTÉTICO.

Por que ela: separar 'provar a esteira' de 'medir o modelo': a demo precisa de determinismo; a qualidade é medida à parte em docs/metricas.md. A cascata é desvio documentado da ADR-0011 (zero IA generativa) ? produção exige NER discriminativo treinado no mesmo gate.

Onde ver: app/motor_ollama.py; docs/metricas.md; ADR-0011.

> *>> na demonstração o motor é o didático ? prova que a esteira inteira funciona. A nota do modelo real está medida no relatório de métricas e ainda não está plugada na porta.*

5. Cola de perguntas ? 'o que o X faz?'

Uma linha por tecnologia, na ordem em que costumam perguntar:

Pergunta	Resposta em uma frase
o que o Vault faz?	guarda a chave E2E fora do código ? os serviços pedem emprestada com token de 60s; dá pra girar e revogar sem deploy
o que o RabbitMQ faz?	a fila interna do nó ? desacopla a API da anonimização pesada; quorum + DLQ
o que o MinIO faz?	o cofre de objetos da instituição ? guarda o documento e o anonimizado, sempre só ciphertext
o que o nginx faz?	a borda mTLS ? exige certificado de instituição e carimba de que órgão é a requisição
o que o Postgres faz?	o livro-razão ? metadados, trilha hash-chain e nonces; sem coluna para bytes de documento
o que o FastAPI faz?	a porta do nó ? recebe tudo e aplica as regras antes de tocar banco ou storage
o que o worker faz?	o anonimizador interno ? consome a fila e publica o objeto tratado, com gate de PII residual
o que o relay faz?	o carteiro outbox ? o que o banco grava, ele garante que chega à fila
o que é o 'E2E'?	quadros ChaCha20-Poly1305 em HTTP, fila e storage ? lacrado + cifrado de ponta a ponta
o que é o Ed25519?	a assinatura da instituição em envelopes e acks ? uma vírgula adulterada invalida tudo
o que é a hash-chain?	cada evento vira sha256(elo anterior | evento) ? mexer num elo quebra todos os seguintes
o que são os nonces?	o carimbo único por pacote ? a fotocópia reenviada é recusada porque o carimbo já foi gasto